

UNICID – Universidade Cidade de São Paulo

Programação Orientada a Objetos: Sistema Acadêmico

São Paulo

2026

Ágatha Ribeiro

Bruno Oliveira Theodoro

Dandhara Fernandes De Campos Lima

Ellen Mayumi Borges

Felipe Neres Vieira

Guilherme dos Santos Matos

Iris Pfister Pascoal

Pedro Fidelis Mandoti

Victor Leandro da Silva

Projeto de Programação Orientada a Objetos, apresentado ao curso de Análise e Desenvolvimento de Sistemas da Universidade Cidade de São Paulo, sob orientação do Prof. Jadir Mendonça

SUMÁRIO

1 INTRODUÇÃO.....	4
2 INTERFACE GRÁFICA.....	5
3 SISTEMA ACADÊMICO.....	7
3.1 AlunoDAO.java.....	7
3.2 CursoDAO.java.....	10
3.3 DisciplinaDAO.java.....	13
3.4 NotaFaltaDAO.java.....	18
4 MODELO.....	24
4.1 Aluno.java.....	24
4.2 Curso.java.....	26
4.3 Disciplina.java.....	27
4.4 NotaFalta.java.....	28
5 BANCO DE DADOS RELACIONAIS MYSQL.....	31
5.1 ConnectionFactory.....	31
5.2 Script MySQL.....	33
6 CONCLUSÃO.....	39

1. Introdução

O desenvolvimento tecnológico vem crescendo cada vez mais ao longo do tempo, beneficiando a globalização, gerando mais oportunidades de emprego, alcançando comunicações entre empresas, influenciando a criatividade, e principalmente, contribuindo para com a educação com inúmeras maneiras de buscar conhecimento na internet, ter acessibilidade a aulas online, pesquisas e consultas a diversos documentos oficiais.

Dessa forma, neste trabalho foi construído um Sistema Acadêmico, inspirado em instituições educacionais, com a inserção da linguagem de programação em Java e MySQL para o banco de dados relacionais, sendo ele utilizado como método de avaliação para a disciplina de programação orientada a objetos.

Em vista disso, o projeto tem como objetivo apresentar o funcionamento do gerenciamento de dados cadastrais e pessoais dos alunos e informações da instituição de ensino, assim como suas ligações com cada curso, disciplina e aula, e consequentemente o lançamento de notas finais ao adicionar funcionalidades que permitem o acesso do usuário a interface gráfica do sistema.

2. Interface gráfica

```
package br.com.academico.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnectionFactory {

    public static Connection getConnection() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            String url    = "jdbc:mysql://localhost:3306/academico";
            String username = "academico";
            String password = "1234"; // se sua instalação do MySQL tiver senha, coloque
aqui
            return DriverManager.getConnection(url, username, password);
        } catch (Exception e) {
            throw new SQLException(e.getMessage());
        }
    }

    public static void closeConnection(Connection conn) throws Exception {
        close(conn);
    }

    private static void close(Connection conn) throws Exception {
```

```
try {  
    if (conn != null) {  
        conn.close();  
    }  
} catch (Exception e) {  
    throw new Exception(e.getMessage());  
}  
}  
}
```

3. Sistema Acadêmico

3.1. AlunoDAO.java

```
package br.com.academico.dao;

import br.com.academico.model.Aluno;
import br.com.academico.modelCurso;
import br.com.academico.util.ConnectionFactory;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Date;

public class AlunoDAO {

    public void salvar(Aluno aluno) throws Exception {
        String sql = "INSERT INTO tb_aluno (rgm, fk_cod_curso, nome, cpf,
data_nascimento, email, endereco, municipio, uf, celular) VALUES (?, ?, ?, ?, ?,
?, ?, ?, ?, ?)";

        try (Connection conn = ConnectionFactory.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, aluno.getRgm());
            stmt.setInt(2, aluno.getCurso().getCodCurso());
            stmt.setString(3, aluno.getNome());
            stmt.setString(4, aluno.getCpf());
            stmt.setDate(5, Date.valueOf(aluno.getDataNascimento()));
            stmt.setString(6, aluno.getEmail());
            stmt.setString(7, aluno.getEndereco());
            stmt.setString(8, aluno.getMunicipio());
            stmt.setString(9, aluno.getUf());
            stmt.setString(10, aluno.getCelular());

            stmt.executeUpdate();
            System.out.println("Aluno " + aluno.getNome() + " cadastrado com
sucesso!");
        }
    }

    public Aluno consultar(int rgm) throws Exception {
        String sql = ""
```

```

SELECT a.*, c.cod_curso, c.nome_curso, c.campus, c.periodo
FROM tb_aluno a
INNER JOIN tb_curso c ON c.cod_curso = a.fk_cod_curso
WHERE a.rgm = ?
""";

```

```

try (Connection conn = ConnectionFactory.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setInt(1, rgm);

    try (ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            Curso curso = new Curso();
            curso.setCodCurso(rs.getInt("cod_curso"));
            curso.setNomeCurso(rs.getString("nome_curso"));
            curso.setCampus(rs.getString("campus"));
            curso.setPeriodo(Curso.Periodo.valueOf(rs.getString("periodo")));

            Aluno alunoEncontrado = new Aluno();
            alunoEncontrado.setRgm(rs.getInt("rgm"));
            alunoEncontrado.setNome(rs.getString("nome"));
            alunoEncontrado.setCpf(rs.getString("cpf"));

            alunoEncontrado.setDataNascimento(rs.getDate("data_nascimento").toLocalDate());

            alunoEncontrado.setEmail(rs.getString("email"));
            alunoEncontrado.setEndereco(rs.getString("endereco"));
            alunoEncontrado.setMunicipio(rs.getString("municipio"));
            alunoEncontrado.setUf(rs.getString("uf"));
            alunoEncontrado.setCelular(rs.getString("celular"));
            alunoEncontrado.setCurso(curso);

            return alunoEncontrado;
        }
    }
}
return null;
}

```

```

public void alterar(Aluno aluno) throws Exception {

```



```
String sql = "UPDATE tb_aluno SET fk_cod_curso = ?, nome = ?, cpf = ?,
data_nascimento = ?, email = ?, endereco = ?, municipio = ?, uf = ?, celular = ?
WHERE rgm = ?";
```

```
try (Connection conn = ConnectionFactory.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setInt(1, aluno.getCurso().getCodCurso());
    stmt.setString(2, aluno.getNome());
    stmt.setString(3, aluno.getCpf());
    stmt.setDate(4, Date.valueOf(aluno.getDataNascimento()));
    stmt.setString(5, aluno.getEmail());
    stmt.setString(6, aluno.getEndereco());
    stmt.setString(7, aluno.getMunicipio());
    stmt.setString(8, aluno.getUf());
    stmt.setString(9, aluno.getCelular());
    stmt.setInt(10, aluno.getRgm());

    stmt.executeUpdate();
    System.out.println("Aluno " + aluno.getNome() + " alterado com
sucesso!");
}
}
```

```
public void excluir(int rgm) throws Exception {
    String sql = "DELETE FROM tb_aluno WHERE rgm = ?";

    try (Connection conn = ConnectionFactory.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, rgm);
        stmt.executeUpdate();
        System.out.println("Aluno de RGM " + rgm + " excluído com
sucesso!");
    }
}
}
```

3.1. CursoDAO.java

```

package br.com.academico.dao;

import br.com.academico.modelCurso;
import br.com.academico.util.ConnectionFactory;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class CursoDAO {

    public void salvar(Curso curso) throws Exception {

        String sql = "INSERT INTO tb_curso (nome_curso, campus, periodo)
VALUES (?, ?, ?)";

        try (Connection conn = ConnectionFactory.getConnection();

            PreparedStatement stmt = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

            stmt.setString(1, curso.getNomeCurso());
            stmt.setString(2, curso.getCampus());
            stmt.setString(3, curso.getPeriodo().name());

            stmt.executeUpdate();

            try (ResultSet rs = stmt.getGeneratedKeys()) {
                if (rs.next()) {
                    curso.setCodCurso(rs.getInt(1));
                }
            }
        }
    }
}

```

```

public List<Curso> listarTodos() throws Exception {

    String sql = "SELECT * FROM tb_curso";

    List<Curso> lista = new ArrayList<>();

    try (Connection conn = ConnectionFactory.getConnection();

        PreparedStatement stmt = conn.prepareStatement(sql);

        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {

            Curso.Periodo periodoEnum =
Curso.Periodo.valueOf(rs.getString("periodo"));

            Curso curso = new Curso(

                rs.getInt("cod_curso"),

                rs.getString("nome_curso"),

                rs.getString("campus"),

                periodoEnum

            );

            lista.add(curso);

        }

    }

    return lista;

}

public Curso buscarPorId(int codCurso) throws Exception {

    String sql = "SELECT * FROM tb_curso WHERE cod_curso = ?";

    try (Connection conn = ConnectionFactory.getConnection();

        PreparedStatement stmt = conn.prepareStatement(sql)) {

```

```

        stmt.setInt(1, codCurso);

        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {

                Curso.Periodo periodoEnum =
                Curso.Periodo.valueOf(rs.getString("periodo"));

```

```

                return new Curso(
                    rs.getInt("cod_curso"),
                    rs.getString("nome_curso"),
                    rs.getString("campus"),
                    periodoEnum
                );
            }
        }
    }
    return null;
}

```

```

public void atualizar(Curso curso) throws Exception {

    String sql = "UPDATE tb_curso SET nome_curso = ?, campus = ?, periodo
= ? WHERE cod_curso = ?";

```

```

    try (Connection conn = ConnectionFactory.getConnection();

        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, curso.getNomeCurso());
        stmt.setString(2, curso.getCampus());
        stmt.setString(3, curso.getPeriodo().name());
        stmt.setInt(4, curso.getCodCurso());

```

```

        stmt.executeUpdate();
    }
}

public void excluir(int codCurso) throws Exception {
    String sql = "DELETE FROM tb_curso WHERE cod_curso = ?";

    try (Connection conn = ConnectionFactory.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, codCurso);
        stmt.executeUpdate();
    }
}
}

```

3.3. DisciplinaDAO.java

```

package br.com.academico.dao;

import br.com.academico.modelCurso;
import br.com.academico.model.Disciplina;
import br.com.academico.util.ConnectionFactory;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class DisciplinaDAO {

    public void salvar(Disciplina disciplina) throws SQLException {

        String sql = "INSERT INTO tb_disciplina (nome_disciplina, fk_cod_curso,
carga_horaria, credits) VALUES (?, ?, ?, ?)";
    }
}

```

```

try (Connection conn = ConnectionFactory.getConnection();

    PreparedStatement stmt = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

    stmt.setString(1, disciplina.getNomeDisciplina());
    stmt.setInt(2, disciplina.getCurso().getCodCurso());

    stmt.setInt(3, 80);
    stmt.setInt(4, 4);

    stmt.executeUpdate();
    try (ResultSet rs = stmt.getGeneratedKeys()) {
        if (rs.next()) {
            disciplina.setCodDisciplina(rs.getInt(1));
        }
    }
}

}

public Disciplina buscarPorId(int codDisciplina) throws SQLException {

    String sql = "SELECT d.cod_disciplina, d.nome_disciplina, c.cod_curso,
c.nome_curso, c.campus, c.periodo " +
        "FROM tb_disciplina d " +
        "JOIN tb_curso c ON d.fk_cod_curso = c.cod_curso " +
        "WHERE d.cod_disciplina = ?";

    try (Connection conn = ConnectionFactory.getConnection();

        PreparedStatement stmt = conn.prepareStatement(sql)) {

```

```

        stmt.setInt(1, codDisciplina);

        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {

                Curso.Periodo periodoEnum =
                Curso.Periodo.valueOf(rs.getString("periodo"));

                Curso curso = new Curso(
                    rs.getInt("cod_curso"),
                    rs.getString("nome_curso"),
                    rs.getString("campus"),
                    periodoEnum
                );
                return new Disciplina(
                    rs.getInt("cod_disciplina"),
                    rs.getString("nome_disciplina"),
                    curso
                );
            }
        }
        return null;
    }

    public List<Disciplina> listarTodos() throws SQLException {
        String sql = "SELECT d.cod_disciplina, d.nome_disciplina, c.cod_curso,
c.nome_curso, c.campus, c.periodo " +
            "FROM tb_disciplina d " +
            "JOIN tb_curso c ON d.fk_cod_curso = c.cod_curso";
    }

```

```

List<Disciplina> lista = new ArrayList<>();

try (Connection conn = ConnectionFactory.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {

        Curso.Periodo periodoEnum =
Curso.Periodo.valueOf(rs.getString("periodo"));

        Curso curso = new Curso(
            rs.getInt("cod_curso"),
            rs.getString("nome_curso"),
            rs.getString("campus"),
            periodoEnum
        );

        lista.add(new Disciplina(
            rs.getInt("cod_disciplina"),
            rs.getString("nome_disciplina"),
            curso
        ));
    }
}

return lista;
}

public void atualizar(Disciplina disciplina) throws SQLException {

    String sql = "UPDATE tb_disciplina SET nome_disciplina = ?,
fk_cod_curso = ? WHERE cod_disciplina = ?";

```



```

try (Connection conn = ConnectionFactory.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setString(1, disciplina.getNomeDisciplina());
    stmt.setInt(2, disciplina.getCurso().getCodCurso());
    stmt.setInt(3, disciplina.getCodDisciplina());

    stmt.executeUpdate();
}
}

public void excluir(int codDisciplina) throws SQLException {
    String sql = "DELETE FROM tb_disciplina WHERE cod_disciplina = ?";

    try (Connection conn = ConnectionFactory.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, codDisciplina);
        stmt.executeUpdate();
    }
}
}

```

3.4. NotaFaltaDAO.java

```

package br.com.academico.dao;

```

```

import br.com.academico.model.Aluno;
import br.com.academico.modelCurso;
import br.com.academico.model.Disciplina;
import br.com.academico.model.NotaFalta;
import br.com.academico.util.ConnectionFactory;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class NotaFaltaDAO {

    public void salvar(NotaFalta notaFalta) throws Exception {

        String sql = "INSERT INTO tb_nota_falta (fk_rgm, fk_cod_disciplina,
        semestre, nota, faltas) VALUES (?, ?, ?, ?, ?)";

        try (Connection conn = ConnectionFactory.getConnection();

            PreparedStatement stmt = conn.prepareStatement(sql,
            Statement.RETURN_GENERATED_KEYS)) {

            stmt.setInt(1, notaFalta.getAluno().getRgm());
            stmt.setInt(2, notaFalta.getDisciplina().getCodDisciplina());
            stmt.setString(3, notaFalta.getSemestre().toString());
            stmt.setDouble(4, notaFalta.getNota());
            stmt.setInt(5, notaFalta.getFaltas());

            stmt.executeUpdate();

            try (ResultSet rs = stmt.getGeneratedKeys()) {
                if (rs.next()) {

```

```

        notaFalta.setCodNotaFalta(rs.getInt(1));
    }
}
}
}

public NotaFalta buscarPorId(int codNotaFalta) throws Exception {
    String sql = "SELECT nf.pk_cod_nota_falta, nf.semestre, nf.nota, nf.faltas,
" +
        "a.rgm AS rgm, a.nome AS nome_aluno, a.cpf, a.data_nascimento,
a.email, a.endereco, a.municipio, a.uf, a.celular, " +
        "d.cod_disciplina, d.nome_disciplina, " +
        "c.cod_curso, c.nome_curso, c.campus, c.periodo " +
        "FROM tb_nota_falta nf " +
        "JOIN tb_aluno a ON nf.fk_rgm = a.rgm " +
        "JOIN tb_disciplina d ON nf.fk_cod_disciplina = d.cod_disciplina " +
        "JOIN tb_curso c ON a.fk_cod_curso = c.cod_curso " +
        "WHERE nf.pk_cod_nota_falta = ?";

    try (Connection conn = ConnectionFactory.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, codNotaFalta);

        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                return montarObjeto(rs);
            }
        }
    }

    return null;
}

```

```
}
```

```
public List<NotaFalta> listarPorAluno(int rgm) throws Exception {
```

```
    String sql = "SELECT nf.pk_cod_nota_falta, nf.semestre, nf.nota, nf.faltas,  
    " +
```

```
        "a.rgm AS rgm, a.nome AS nome_aluno, a.cpf, a.data_nascimento,  
a.email, a.endereco, a.municipio, a.uf, a.celular, " +
```

```
        "d.cod_disciplina, d.nome_disciplina, " +
```

```
        "c.cod_curso, c.nome_curso, c.campus, c.periodo " +
```

```
        "FROM tb_nota_falta nf " +
```

```
        "JOIN tb_aluno a ON nf.fk_rgm = a.rgm " +
```

```
        "JOIN tb_disciplina d ON nf.fk_cod_disciplina = d.cod_disciplina " +
```

```
        "JOIN tb_curso c ON a.fk_cod_curso = c.cod_curso " +
```

```
        "WHERE nf.fk_rgm = ? ORDER BY nf.semestre ASC";
```

```
    List<NotaFalta> lista = new ArrayList<>();
```

```
    try (Connection conn = ConnectionFactory.getConnection();
```

```
        PreparedStatement stmt = conn.prepareStatement(sql)) {
```

```
        stmt.setInt(1, rgm);
```

```
        try (ResultSet rs = stmt.executeQuery()) {
```

```
            while (rs.next()) {
```

```
                lista.add(montarObjeto(rs));
```

```
            }
```

```
        }
```

```
    }
```

```
    return lista;
```

```
}
```

```

public void atualizar(NotaFalta notaFalta) throws Exception {

    String sql = "UPDATE tb_nota_falta SET fk_rgm = ?, fk_cod_disciplina =
?, semestre = ?, nota = ?, faltas = ? " +

        "WHERE pk_cod_nota_falta = ?";

    try (Connection conn = ConnectionFactory.getConnection();

        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, notaFalta.getAluno().getRgm
        stmt.setInt(2, notaFalta.getDisciplina().getCodDisciplina());
        stmt.setString(3, notaFalta.getSemestre().toString());
        stmt.setDouble(4, notaFalta.getNota());
        stmt.setInt(5, notaFalta.getFaltas());
        stmt.setInt(6, notaFalta.getCodNotaFalta());

        stmt.executeUpdate();

    }

}

public void excluir(int codNotaFalta) throws Exception {

    String sql = "DELETE FROM tb_nota_falta WHERE pk_cod_nota_falta =
?";

    try (Connection conn = ConnectionFactory.getConnection();

        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, codNotaFalta);

        stmt.executeUpdate();

    }

}

```

```

private NotaFalta.Semestre converterStringParaEnum(String textoBanco) {
    for (NotaFalta.Semestre sem : NotaFalta.Semestre.values()) {
        if (sem.toString().equals(textoBanco)) {
            return sem;
        }
    }
    throw new IllegalArgumentException("Semestre inválido no banco: " +
    textoBanco);
}

```

```

private NotaFalta montarObjeto(ResultSet rs) throws Exception {
    Curso.Periodo periodoEnum =
    Curso.Periodo.valueOf(rs.getString("periodo"));
    Curso curso = new Curso(
        rs.getInt("cod_curso"),
        rs.getString("nome_curso"),
        rs.getString("campus"),
        periodoEnum
    );
}

```

```

Aluno aluno = new Aluno();
aluno.setRgm(rs.getInt("rgm"));
aluno.setNome(rs.getString("nome_aluno"));
aluno.setCurso(curso);
aluno.setCpf(rs.getString("cpf"));
aluno.setEmail(rs.getString("email"));
}

```

```

Disciplina disciplina = new Disciplina(
    rs.getInt("cod_disciplina"),
}

```

```

        rs.getString("nome_disciplina"),
        curso
    );

    NotaFalta.Semestre semestreEnum =
    converterStringParaEnum(rs.getString("semestre"));

    return new NotaFalta(
        rs.getInt("pk_cod_nota_falta"),
        aluno,
        disciplina,
        semestreEnum,
        rs.getDouble("nota"),
        rs.getInt("faltas")
    );
}
}

```

4. Modelo

4.1. Aluno.java

```

package br.com.academico.model;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

```

```

public class Aluno {
    .
    private int rgm;
    private String nome;
    private String cpf;
    private LocalDate dataNascimento;
    private String email;
    private String endereco;
    private String municipio;
    private String uf;
    private String celular;

    private Curso curso;
    private List<NotaFalta> notasFaltas;

    public Aluno() {
        this.notasFaltas = new ArrayList<>();
    }

    public Aluno(int rgm, String nome, String cpf, LocalDate dataNascimento,
        String email, String endereco, String municipio,
        String uf, String celular) {

        this.rgm = rgm;
        this.nome = nome;
        this.cpf = cpf;
        this.dataNascimento = dataNascimento;
        this.email = email;
        this.endereco = endereco;
        this.municipio = municipio;
        this.uf = uf;
        this.celular = celular;

        this.notasFaltas = new ArrayList<>();
    }

    public void adicionarNotaFalta(NotaFalta nf) {
        if (nf != null) {
            notasFaltas.add(nf);
        }
    }

    public void removerNotaFalta(NotaFalta nf) {

```



```

        notasFaltas.remove(nf);
    }

    // getters e setters (rgm agora int)
    public int getRgm() { return rgm; }
    public void setRgm(int rgm) { this.rgm = rgm; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getCpf() { return cpf; }
    public void setCpf(String cpf) { this.cpf = cpf; }

    public LocalDate getDataNascimento() { return dataNascimento; }
    public void setDataNascimento(LocalDate dataNascimento) {
this.dataNascimento = dataNascimento; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getEndereco() { return endereco; }
    public void setEndereco(String endereco) { this.endereco = endereco; }

    public String getMunicipio() { return municipio; }
    public void setMunicipio(String municipio) { this.municipio = municipio; }

    public String getUf() { return uf; }
    public void setUf(String uf) { this.uf = uf; }

    public String getCelular() { return celular; }
    public void setCelular(String celular) { this.celular = celular; }

    public Curso getCurso() { return curso; }
    public void setCurso(Curso curso) { this.curso = curso; }

    public List<NotaFalta> getNotasFaltas() { return notasFaltas; }
    public void setNotasFaltas(List<NotaFalta> notasFaltas) { this.notasFaltas =
notasFaltas; }

    @Override
    public String toString() {
        return "model.Aluno{rgm=" + rgm + ", nome=" + nome + "}";
    }

```

```
}
```

4.2. Curso.java

```
package br.com.academico.model;
```

```
public class Curso {  
    public enum Periodo {  
        MATUTINO,  
        VESPERTINO,  
        NOTURNO;  
  
        @Override  
        public String toString() {  
            return name().charAt(0) + name().substring(1).toLowerCase();  
        }  
    }  
  
    private int codCurso;  
    private String nomeCurso;  
    private String campus;  
    private Periodo periodo;  
  
    public Curso() {}  
  
    public Curso(int codCurso, String nomeCurso, String campus, Periodo  
periodo) {  
        this.codCurso = codCurso;  
        this.nomeCurso = nomeCurso;  
        this.campus = campus;  
        this.periodo = periodo;  
    }  
  
    public int getCodCurso() { return codCurso; }  
  
    public void setCodCurso(int codCurso) { this.codCurso = codCurso; }  
  
    public String getNomeCurso() { return nomeCurso; }  
  
    public void setNomeCurso(String nomeCurso) { this.nomeCurso =  
nomeCurso; }  
}
```

```

public String getCampus() { return campus; }

public void setCampus(String campus) { this.campus = campus; }

public Periodo getPeriodo() { return periodo; }

public void setPeriodo(Periodo periodo) { this.periodo = periodo; }

@Override
public String toString() {
    return nomeCurso;
}
}

```

4.3. Disciplina.java

```

package br.com.academico.model;

public class Disciplina {
    private int codDisciplina;
    private String nomeDisciplina;
    private Curso curso;

    public Disciplina(int codDisciplina, String nomeDisciplina, Curso curso) {
        this.codDisciplina = codDisciplina;
        this.nomeDisciplina = nomeDisciplina;
        this.curso = curso;
    }

    public int getCodDisciplina() { return codDisciplina; }
    public void setCodDisciplina(int codDisciplina) { this.codDisciplina =
codDisciplina; }

    public String getNomeDisciplina() { return nomeDisciplina; }
    public void setNomeDisciplina(String nomeDisciplina) { this.nomeDisciplina
= nomeDisciplina; }

    public Curso getCurso() { return curso; }
    public void setCurso(Curso curso) { this.curso = curso; }

    @Override

```

```

    public String toString() {
        return nomeDisciplina;
    }
}

```

4.4. NotaFalta.java

```

package br.com.academico.model;

public class NotaFalta {
    // model.NotaFalta representa o registro de nota e falta de UM aluno em UMA
    disciplina
    // Relacionamento:
    //  pertence a UM aluno    (FK: rgm)      → ON DELETE CASCADE
    //  pertence a UMA disciplina (FK: codDisciplina)
    // Requisito do professor: "se um aluno for EXCLUIDO suas notas e faltas
    também deverão ser excluídas"
    // enum Semestre — mesmo padrão do enum Período dentro do model.Curso
    // define um conjunto FIXO de semestres válidos para evitar valores inválidos
    // cobre desde 2020 conforme exemplo do PDF (ex: 2020-1)

    public enum Semestre {
        S1_2020("2020-1"),
        S2_2020("2020-2"),
        S1_2021("2021-1"),
        S2_2021("2021-2"),
        S1_2022("2022-1"),
        S2_2022("2022-2"),
        S1_2023("2023-1"),
        S2_2023("2023-2"),
        S1_2024("2024-1"),
        S2_2024("2024-2"),
        S1_2025("2025-1"),
        S2_2025("2025-2"),
        S1_2026("2026-1"),
        S2_2026("2026-2");

        private final String descricao;

        Semestre(String descricao) {
            this.descricao = descricao;
        }
    }
}

```

```

        @Override
        public String toString() {
            return descricao;
        }
    }

    private int codNotaFalta;
    private Aluno aluno;
    private Disciplina disciplina;
    private Semestre semestre;
    private double nota;
    private int faltas;

    public NotaFalta() {}
    public NotaFalta(int codNotaFalta, Aluno aluno, Disciplina disciplina,
        Semestre semestre, double nota, int faltas) {
        this.codNotaFalta = codNotaFalta;
        this.aluno = aluno;
        this.disciplina = disciplina;
        this.semestre = semestre;
        this.nota = nota;
        this.faltas = faltas;
    }

    public int getCodNotaFalta() { return codNotaFalta; }
    public void setCodNotaFalta(int codNotaFalta) { this.codNotaFalta =
codNotaFalta; }

    public Aluno getAluno() { return aluno; }
    public void setAluno(Aluno aluno) { this.aluno = aluno; }

    public Disciplina getDisciplina() { return disciplina; }
    public void setDisciplina(Disciplina disciplina) { this.disciplina = disciplina;
}

    public Semestre getSemestre() { return semestre; }
    public void setSemestre(Semestre semestre) { this.semestre = semestre; }

    public double getNota() { return nota; }
    public void setNota(double nota) { this.nota = nota; }

    public int getFaltas() { return faltas; }

```

```
public void setFaltas(int faltas) { this.faltas = faltas; }
```

```
@Override
```

```
public String toString() {
```

```
    return "model.NotaFalta{" +
```

```
        "aluno=" + aluno.getNome() +
```

```
        ", disciplina=" + disciplina.getNomeDisciplina() +
```

```
        ", semestre=" + semestre +
```

```
        ", nota=" + nota +
```

```
        ", faltas=" + faltas + "}";
```

```
    }
```

```
}
```

5. Banco de Dados relacionais MySQL

5.1. ConnectionFactory

```
package br.com.academico.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnectionFactory {

    public static Connection getConnection() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            String url    = "jdbc:mysql://localhost:3306/academico";
            String username = "academico";
            String password = "1234"; // se sua instalação do MySQL tiver senha,
coloque aqui

            return DriverManager.getConnection(url, username, password);
        } catch (Exception e) {
            throw new SQLException(e.getMessage());
        }
    }

    public static void closeConnection(Connection conn) throws Exception {
        close(conn);
    }
}
```

```

private static void close(Connection conn) throws Exception {
    try {
        if (conn != null) {
            conn.close();
        }
    } catch (Exception e) {
        throw new Exception(e.getMessage());
    }
}
}

```

5.2. Script MySQL

```

CREATE TABLE tb_curso (
    cod_curso int NOT NULL AUTO_INCREMENT,
    nome_curso enum(
        'ANALISE E DESENVOLVIMENTO DE SISTEMAS',
        'CIENCIA DA COMPUTACAO',
        'ENGENHARIA DE SOFTWARE'
    ) NOT NULL,
    campus varchar(30) NOT NULL,

```



```

    periodo enum('MATUTINO','VESPERTINO','NOTURNO') NOT NULL,
    created_at datetime DEFAULT CURRENT_TIMESTAMP,
    updated_at datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
    UNIQUE INDEX uq_curso_horario_campus (nome_curso, campus, periodo),
    CONSTRAINT tb_curso_pk PRIMARY KEY (cod_curso)
);

```

```

CREATE TABLE tb_aluno (
    rgm          char(8)    NOT NULL CHECK (rgm REGEXP '^[0-9]{8}$'),
    fk_cod_curso  int        NOT NULL,
    nome         varchar(120) NOT NULL,
    cpf          varchar(14) NOT NULL CHECK (cpf REGEXP '^[0-9]{3}\\. [0-9]{3}\\. [0-9]{3}-[0-9]{2}$'),
    data_nascimento date     NOT NULL,
    email        varchar(100) NOT NULL,
    endereco     varchar(150) NOT NULL,
    municipio    varchar(80)  NOT NULL,
    uf           char(2)     NOT NULL,
    celular      varchar(15)  NOT NULL CHECK (celular REGEXP '^\\([0-9]{2}\\)[0-9]{4,5}-[0-9]{4}$'),
    created_at datetime DEFAULT CURRENT_TIMESTAMP,
    updated_at datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
    UNIQUE INDEX uq_cpf (cpf),
    CONSTRAINT tb_aluno_pk PRIMARY KEY (rgm)
);

```

```

CREATE TABLE tb_disciplina (
    cod_disciplina int NOT NULL AUTO_INCREMENT,

```

```

nome_disciplina enum(
    'PROGRAMACAO ORIENTADA A OBJETOS',
    'BANCO DE DADOS',
    'CALCULO DIFERENCIAL E INTEGRAL II',
    'ESTRUTURA DE DADOS'
) NOT NULL,

fk_cod_curso int NOT NULL,

carga_horaria int NOT NULL,

creditos int NOT NULL,

created_at datetime DEFAULT CURRENT_TIMESTAMP,

updated_at datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

CONSTRAINT tb_disciplina_pk PRIMARY KEY (cod_disciplina)
);

```

```

CREATE TABLE tb_nota_falta (
    pk_cod_nota_falta int NOT NULL AUTO_INCREMENT,
    fk_rgm char(8) NOT NULL,
    fk_cod_disciplina int NOT NULL,
    semestre varchar(6) NOT NULL,
    nota decimal(4,2) NOT NULL CHECK (nota >= 0.00 AND nota <= 10.00),
    faltas int NOT NULL CHECK (faltas >= 0),
    created_at datetime DEFAULT CURRENT_TIMESTAMP,
    updated_at datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    UNIQUE INDEX uq_aluno_disciplina_semestre (fk_rgm, fk_cod_disciplina,
semestre),

    INDEX idx_nota_falta_rgm (fk_rgm),
    INDEX idx_nota_falta_disciplina (fk_cod_disciplina),
    CONSTRAINT tb_nota_falta_pk PRIMARY KEY (pk_cod_nota_falta)

```

);

```
ALTER TABLE tb_aluno ADD CONSTRAINT fk_id_curso_aluno  
    FOREIGN KEY fk_id_curso_aluno (fk_cod_curso)  
    REFERENCES tb_curso (cod_curso)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE;
```

```
ALTER TABLE tb_disciplina ADD CONSTRAINT fk_id_curso_disciplina  
    FOREIGN KEY fk_id_curso_disciplina (fk_cod_curso)  
    REFERENCES tb_curso (cod_curso)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE;
```

```
ALTER TABLE tb_nota_falta ADD CONSTRAINT fk_id_disciplina_nota_falta  
    FOREIGN KEY fk_id_disciplina_nota_falta (fk_cod_disciplina)  
    REFERENCES tb_disciplina (cod_disciplina)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE;
```

```
ALTER TABLE tb_nota_falta ADD CONSTRAINT fk_rgm_nota_falta  
    FOREIGN KEY fk_rgm_nota_falta (fk_rgm)  
    REFERENCES tb_aluno (rgm)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE;
```

DELIMITER \$\$

-- Valida data_nascimento no INSERT (CURRENT_DATE não é suportado em CHECK no MySQL)

CREATE TRIGGER trg_valida_nascimento_insert

BEFORE INSERT ON tb_aluno

FOR EACH ROW

BEGIN

IF NEW.data_nascimento > CURDATE() THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'data_nascimento não pode ser uma data futura.';

END IF;

END\$\$

-- Valida data_nascimento no UPDATE

CREATE TRIGGER trg_valida_nascimento_update

BEFORE UPDATE ON tb_aluno

FOR EACH ROW

BEGIN

IF NEW.data_nascimento > CURDATE() THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'data_nascimento não pode ser uma data futura.';

END IF;

END\$\$

-- Valida semestre (formato YYYY-1 ou YYYY-2) e arredonda nota no INSERT

-- Arredondamento: CEIL(nota - 0.25) → >= x.75 sobe para o inteiro seguinte

-- Ex.: 1.75 → CEIL(1.50) = 2 | 1.74 → CEIL(1.49) = 1

```

CREATE TRIGGER trg_nota_falta_insert
BEFORE INSERT ON tb_nota_falta
FOR EACH ROW
BEGIN
    IF NEW.semestre NOT REGEXP '^[0-9]{4}-[12]$\'' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Semestre inválido. Use o formato YYYY-1 ou YYYY-2.';
    END IF;

    IF NEW.nota < 0 OR NEW.nota > 10 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Nota deve estar entre 0 e 10.';
    END IF;

    SET NEW.nota = CEIL(NEW.nota - 0.25);
END$$

-- Valida semestre e arredonda nota no UPDATE
CREATE TRIGGER trg_nota_falta_update
BEFORE UPDATE ON tb_nota_falta
FOR EACH ROW
BEGIN
    IF NEW.semestre NOT REGEXP '^[0-9]{4}-[12]$\'' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Semestre inválido. Use o formato YYYY-1 ou YYYY-2.';
    END IF;

    IF NEW.nota < 0 OR NEW.nota > 10 THEN

```

```

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Nota deve estar entre 0 e 10.';

    END IF;

    SET NEW.nota = CEIL(NEW.nota - 0.25);

END$$

DELIMITER ;

CREATE VIEW vw_boletim AS
SELECT
    a.rgm,
    a.nome,
    c.nome_curso,
    d.nome_disciplina,
    nf.semestre,
    nf.nota,
    nf.faltas,
    CASE
        WHEN nf.nota >= 5.00 AND nf.faltas <= 15 THEN 'Aprovado'
        ELSE 'Reprovado'
    END AS situacao
FROM tb_nota_falta nf
JOIN tb_aluno a ON a.rgm = nf.fk_rgm
JOIN tb_disciplina d ON d.cod_disciplina = nf.fk_cod_disciplina
JOIN tb_curso c ON c.cod_curso = a.fk_cod_curso;
.
```

6. Conclusão

Este projeto teve como objetivo a construção de um Sistema Acadêmico para o gerenciamento de dados dos alunos de uma instituição educacional, nele foi utilizada a linguagem de programação em Java, inserindo as quatro principais classes abordadas no programa, a biblioteca gráfica Swing e MySQL para o banco de dados relacionais, utilizando campos que identificaram cada dado, tal qual chave primária e chave estrangeira, exclusivamente para a avaliação da disciplina de Programação Orientada a Objetos.

Por conseguinte, é possível observar na prática as funcionalidades que foram resultado do programa na visão do usuário, como a restrição e encapsulamento de material privado, consulta de dados pessoais, informações importantes do estabelecimento, lançamento de notas no sistema e atualizações de frequência e ausência do aluno por meio da interface gráfica.

Dessa maneira, a finalidade proposta pela avaliação foi alcançada não somente referente a consultas, como também na realização de cadastros, alterações, atualizações e exclusões de dados.